

Lezione 22 - Spring Rest

Introduzione : da JDBC a Spring

Fino ad ora abbiamo visto come JDBC permetta di comunicare con un database in modo diretto ed esplicito:

- si apre una connessione,
- si scrive la query SQL,
- si itera sul `ResultSet`,
- si chiude tutto.

Funziona, ma in un'applicazione reale questo approccio diventa rapidamente **verboso e ripetitivo** — **ogni operazione richiede decine di righe di codice per gestire connessioni, eccezioni e risorse.**

Inoltre JDBC lavora a **basso livello**:

- **sei tu a scrivere ogni query SQL, a mappare ogni riga del `ResultSet` su un oggetto Java, a gestire le transazioni manualmente.**

In un'applicazione enterprise con decine di entità e centinaia di operazioni, questo diventa insostenibile.

È qui che entra in gioco **Spring**:

- **un framework che astrae tutta questa complessità, permettendo di concentrarsi sulla logica applicativa invece che sulla gestione dell'infrastruttura.**

Spring non sostituisce JDBC completamente:

- **lo usa internamente sollevandoci dalla responsabilità di gestirlo direttamente.**

Studiare JDBC prima di Spring non è quindi tempo sprecato:

- **è esattamente la base concettuale su cui Spring costruisce le sue astrazioni.**

Le Applicazioni Distribuite

Prima di entrare in Spring, è necessario capire il contesto architetturale in cui opera.

Le applicazioni moderne non sono più monolitiche — sono **applicazioni distribuite**:

- **sistemi in cui i vari componenti si trovano su macchine distinte e cooperano scambiandosi dati e messaggi.**

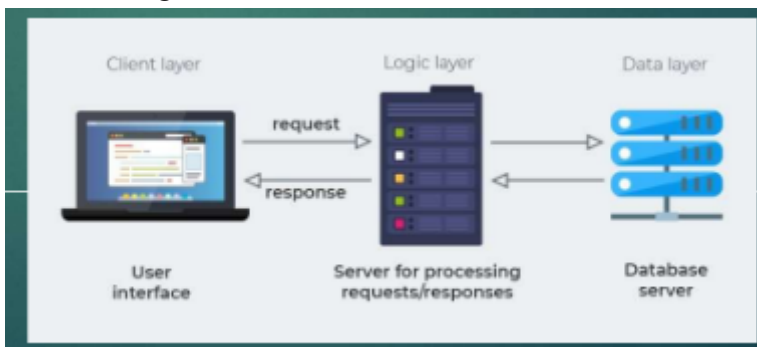
≡ Un esempio concreto:

quando si scarica l'app mobile di Amazon, non si scaricano tutte le dipendenze di Amazon — si scarica solo l'interfaccia utente, che comunica con server sparsi in diverse parti del mondo. Questa separazione non è casuale, ma risponde a esigenze di **scalabilità e disponibilità**: se un server è sovraccarico, se ne aggiunge un altro; se uno cade, gli altri continuano a funzionare.

Per gestire questa complessità, le applicazioni distribuite vengono organizzate in **layer** — strati con responsabilità separate. L'architettura più diffusa è la **three-tier**:

1. **Client Layer** — **l'interfaccia utente (browser, app mobile)**
2. **Logic Layer** — **la logica applicativa, ovvero il cuore dell'applicazione**
3. **Data Layer** — **il database e la persistenza dei dati**

Nelle prossime lezioni ci occuperemo principalmente del **Logic Layer**, simulando tutti e tre gli strati sulla nostra macchina locale.



1. Client Layer — Il Livello di Presentazione

Il Client Layer è il livello più vicino all'utente:

- **il suo scopo principale è visualizzare le informazioni e raccogliere i dati tramite l'interazione con l'utente.**

Può assumere forme diverse:

- **Browser web** — **l'utente accede all'applicazione tramite un URL**

- **Applicazione desktop** — una GUI (interfaccia grafica a finestre) installata sulla macchina
- **Applicazione mobile** — come nell'esempio di Amazon che abbiamo visto

In tutti i casi, il Client Layer non contiene logica applicativa né accede direttamente al database — **si limita a presentare i dati che riceve dal Logic Layer e a inviare le richieste dell'utente verso di esso.**

Nel contesto delle applicazioni web, questo livello è tipicamente sviluppato con il trio classico:

- **HTML** — struttura della pagina
- **CSS** — stile e presentazione visiva
- **JavaScript** — interattività lato client

🔗 È importante sottolineare che il Client Layer è l'unico strato con cui l'utente interagisce direttamente — tutto ciò che accade nei layer sottostanti (logica e dati) è completamente trasparente per lui.

2. Logic Layer — Il Livello Applicativo

Il Logic Layer è il cuore dell'applicazione:

- **riceve le richieste dal Client Layer, le elabora applicando la logica di business, e interagisce con il Data Layer per leggere o modificare i dati.**

📋 **La logica di business è l'insieme delle regole e dei comportamenti specifici del sistema**

ad esempio: "un utente non può acquistare un prodotto se non è autenticato", oppure "il prezzo finale deve includere l'IVA".

Queste regole non appartengono né all'interfaccia utente né al database — **vivono nel Logic Layer.**

Le operazioni che può compiere sul Data Layer sono:

- **Leggere dati per restituirli al client**
- **Aggiungere nuovi dati**
- **Modificare dati esistenti**

- **Eliminare dati**

La comunicazione con il Data Layer avviene tramite **librerie e API** — ed è esattamente il ruolo che abbiamo visto svolgere a JDBC e, come vedremo, a Spring.

Questo livello è tipicamente sviluppato con linguaggi come **Java, Python, PHP, Ruby o Perl**. Nel nostro caso useremo **Java con Spring**.

🔗 **È importante notare il collegamento con quanto già visto:**

il **'LibroDAO'** che abbiamo scritto con JDBC è a tutti gli effetti un componente del Logic Layer — **contiene la logica di accesso ai dati e fa da ponte tra l'applicazione e il database.**

3. Data Layer — Il Livello di Persistenza

Il Data Layer è il livello più basso dell'architettura three-tier:

- **il suo scopo è archiviare e gestire i dati persistenti dell'applicazione.**

Interagisce esclusivamente con il Logic Layer — non comunica mai direttamente con il Client Layer.

Questo livello si realizza tipicamente con:

- **Database relazionali** — PostgreSQL, MySQL, MariaDB, Oracle, Microsoft SQL Server
- **Database NoSQL** — Cassandra, CouchDB, MongoDB (per strutture dati più flessibili)

JDBC, Hibernate e Spring — L'evoluzione della persistenza

JDBC, come abbiamo visto, opera a **basso livello**:

- **il programmatore scrive direttamente le query SQL e gestisce manualmente la mappatura tra righe del database e oggetti Java.**

Questo lo rende didatticamente prezioso — capire JDBC significa capire cosa succede "sotto il cofano" — ma in un'applicazione enterprise diventa rapidamente verboso e ripetitivo.

Hibernate risolve questo problema:

- il programmatore scrive le classi Java e Hibernate si occupa di generare le query SQL in background, usando JDBC internamente.
- Non si "sconfina" più nel SQL — si lavora esclusivamente con oggetti Java.
- Questo approccio si chiama ORM (Object Relational Mapping).

Spring va ancora oltre, integrando Hibernate e aggiungendo un'infrastruttura completa per lo sviluppo enterprise.

Nota sul corso:

Per simulare il Data Layer nelle prossime lezioni useremo le mappe Java (`Map<K, V>`) al posto di un database reale.

Le mappe, come abbiamo già visto, sono strutture chiave-valore che ricordano sia la struttura di un JSON che quella di una tabella SQL — ovviamente con molta meno potenza, ma sufficienti per simulare la persistenza dei dati durante lo sviluppo del Logic Layer con Spring.

Applicazioni Client-Server

Un sistema client-server, come abbiamo già visto diverse volte, è:

- un modello di rete in cui un programma detto **CLIENT** richiede servizi a un altro programma detto **SERVER**.
Client e server girano tipicamente su macchine distinte collegate in rete, anche se — come faremo durante il corso — è possibile simulare entrambi sulla stessa macchina locale.

Le regole generali sono tre:

1. Il **server** eroga servizi su richiesta
2. Il **client** si collega al server e richiede servizi
3. Client e server devono aver concordato un **protocollo di comunicazione**

Un esempio chiarificatore è Gmail:

quando arriva una nuova email si ha la sensazione che sia il server ad agire di sua iniziativa.

In realtà è l'applicazione Gmail che, in background, continua ad interrogare il

server a intervalli regolari fino a quando non riceve nuovi dati. Il server non "spinge" mai dati al client — è sempre il client che li va a chiedere.

Le Applicazioni Web

Le applicazioni web sono **particolari applicazioni distribuite** che adottano l'architettura client-server e comunicano tramite il protocollo **TCP/IP**.

Si dividono in due famiglie principali:

1. Applicazioni a pagine:

- il client è l'utente che, tramite browser, richiede un servizio al server e riceve in risposta una **pagina web**.

☰ Esempio:

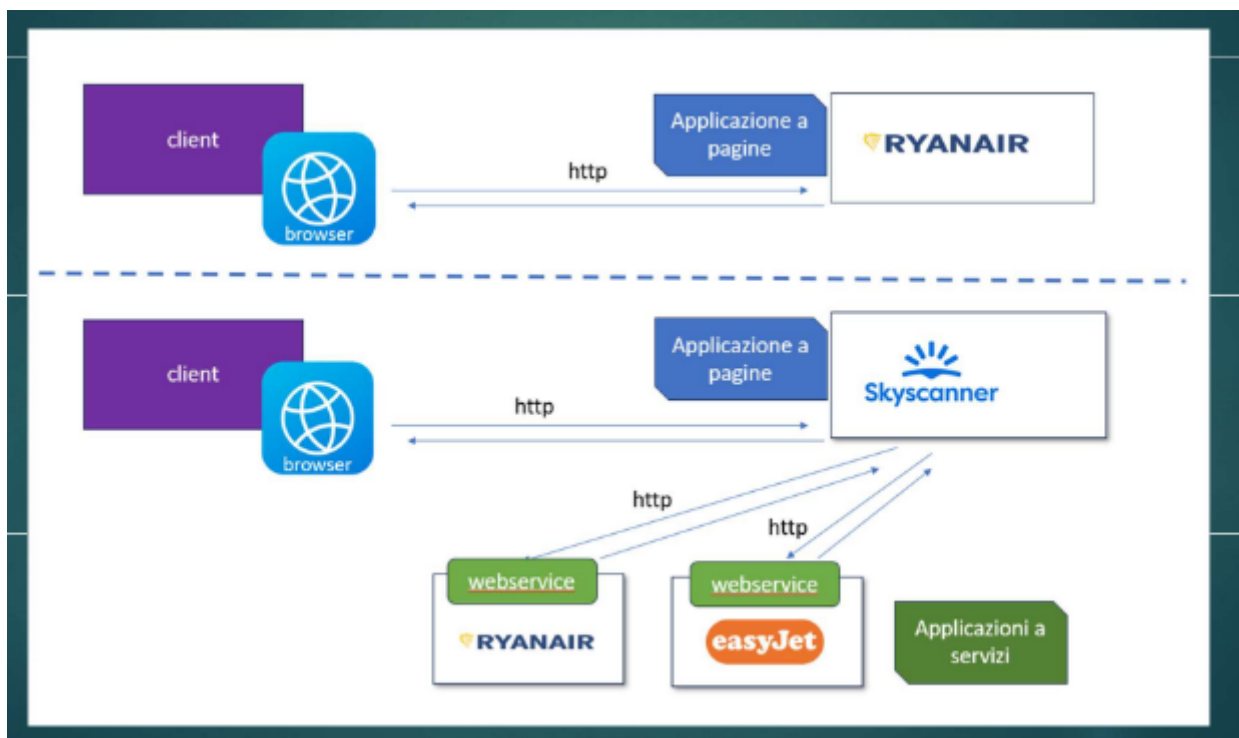
L'utente si collega al sito di Ryanair, naviga tra le pagine e visualizza i voli disponibili.

2. Applicazioni a servizi:

- **il client è un programma che richiede un servizio al server e riceve in risposta dati strutturati (tipicamente JSON), non pagine web.**

☰ Esempio:

Skyscanner non ha i voli di Ryanair, EasyJet o altre compagnie nel suo database — fa da intermediario, interrogando i server delle singole compagnie e aggregando i risultati per mostrarli all'utente. In questo caso Skyscanner è esso stesso un client che parla con altri server.



🔗 Questa distinzione è fondamentale per capire cosa costruiremo con Spring:

- non applicazioni a pagine, ma applicazioni a servizi — componenti del Logic Layer che espongono dati in formato JSON e comunicano con altri programmi, non direttamente con l'utente finale.

Caratteristiche di un'Applicazione a Servizi

Ora che abbiamo capito cosa distingue un'applicazione a servizi da una a pagine, vale la pena analizzarne le caratteristiche nel dettaglio.

1. Il client è un programma:

- Quindi nelle applicazioni a pagine il client è l'utente che interagisce tramite browser.
- Nelle applicazioni a servizi invece il client è sempre un **programma**;
 - può essere un'app Android o iOS, un browser che esegue codice JavaScript, o addirittura un altro server.
 - Questo è esattamente il caso di Skyscanner che abbiamo visto prima: Skyscanner è allo stesso tempo server per il browser dell'utente e client per i server di Ryanair e EasyJet.

2. Client e server si scambiano solo dati:

- Non vengono scambiate pagine HTML, né vengono scambiate interfacce grafiche:
 - **solo dati puri, tipicamente in formato JSON.**
 - **È il client che decide come presentare quei dati all'utente finale.**
 - Questo è il motivo per cui la stessa applicazione backend può servire contemporaneamente un'app Android, un'app iOS e un browser web:
 - **tutti ricevono gli stessi dati JSON e ognuno li visualizza a modo suo.**

3. Il sistema è interoperabile:

- **Poiché client e server si scambiano solo dati tramite protocolli standard, possono essere scritti con linguaggi e tecnologie completamente diverse e funzionare comunque insieme.**
- Il server può essere scritto in Java con Spring, il client in JavaScript, Swift o Kotlin — non importa. **Questa interoperabilità è uno dei motivi principali per cui le applicazioni a servizi hanno dominato lo sviluppo moderno.**

4. SOAP vs REST:

- Esistono due stili principali per realizzare applicazioni a servizi.
- **SOAP:**
 - **è lo stile classico, più rigido e verboso, basato su XML — ancora presente in sistemi legacy ma sempre meno usato nelle nuove applicazioni.**
- **REST:**
 - **è il nuovo stile, più leggero e flessibile, basato su HTTP e JSON — ed è diventato lo standard de facto per le applicazioni moderne.**

👁 **Noi useremo REST con Spring — è lo stile che vedremo nel dettaglio nelle prossime sezioni.**

Il Protocollo TCP/IP

Un protocollo è:

- **semplicemente un insieme di regole concordate per dialogare**
- **esattamente come nella comunicazione tra client e server che abbiamo visto, dove entrambi devono parlare la stessa lingua per capirsi.**

Il **TCP/IP** è il protocollo fondamentale del web. Il suo principio base è semplice ma fondamentale:

- **è sempre il client ad invocare il server, mai il contrario** — abbiamo già visto questo concetto con l'esempio di Gmail.

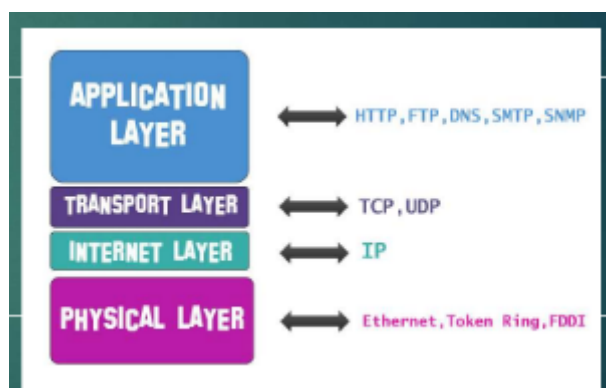
TCP/IP non è un protocollo singolo, ma una **famiglia di protocolli organizzata in 4 livelli**.

Ogni livello si occupa di un aspetto specifico della comunicazione e prevede un proprio set di regole.

La comunicazione funziona correttamente solo se **entrambi i lati usano lo stesso sotto-protocollo per ciascun livello** — esattamente come due persone che devono parlare la stessa lingua a ogni livello della conversazione.

I **4 livelli**, dal più basso al più alto, sono:

1. **Livello fisico** — **gestisce la trasmissione dei dati sul mezzo fisico (cavi, onde radio, ecc.) tramite il protocollo nativo dell'infrastruttura di rete.**
2. **Livello rete** — si occupa dell'instradamento dei dati tra macchine diverse tramite il protocollo **IP** (Internet Protocol). È qui che vivono gli indirizzi IP.
3. **Livello trasporto** — **gestisce la trasmissione affidabile dei dati tramite TCP (affidabile, garantisce la consegna) o UDP (più veloce ma senza garanzie di consegna).**
4. **Livello applicazione** — **è il livello più vicino al programmatore e prevede diversi protocolli specifici per esigenze diverse :**
 - **HTTP** — per il trasferimento di ipertesto, ovvero le pagine web e i dati JSON
 - **FTP** — per il trasferimento di file di grandi dimensioni
 - **SMTP** — per l'invio di email



👁 Noi lavoreremo principalmente a livello applicazione con **HTTP** — il protocollo su cui si basa **REST** e quindi tutto ciò che costruiremo con

Il Livello Applicazione e HTTP

Quando si sviluppa un'applicazione web, ci si occupa esclusivamente del **livello applicazione** — i livelli sottostanti (fisico, rete, trasporto) sono gestiti dall'infrastruttura di rete e sono trasparenti al programmatore.

Per le applicazioni a pagine esiste da sempre il protocollo **HTTP**.

Per le applicazioni a servizi non esisteva un protocollo dedicato — si è quindi deciso di riutilizzare HTTP anche per questo scopo. È per questo che sia le pagine web che le **API REST** usano lo stesso protocollo.

Il Protocollo HTTP

Come abbiamo già detto, il funzionamento di HTTP è semplice:

- **il server è in ascolto in attesa di richieste, il client invia una richiesta specificando due elementi fondamentali:**

1. L'**URL** — **identifica la risorsa con cui si vuole interagire**, seguendo questo formato:



HTTP

```
1 http://IP:port/path/resource
```

Ad esempio:



HTTP

```
1 http://localhost:8080/api/libri
```

2. Il **VERB** — una parola codificata dal protocollo che indica il **tipo di azione** che si vuole compiere sulla risorsa.

In ambito **REST** si usano i seguenti:

Verb	Operazione
GET	Lettura
POST	Inserimento
PUT	Modifica totale

Verb	Operazione
PATCH	Modifica parziale
DELETE	Cancellazione

🔗 Nota:

il parallelismo con le operazioni che abbiamo già visto in [JDBC](#):

- ``executeQuery()`` corrisponde a GET ,
 - mentre ``executeUpdate()`` copre POST , PUT , PATCH e DELETE .
- In REST questi concetti prendono un nome standard e vengono esposti tramite HTTP.

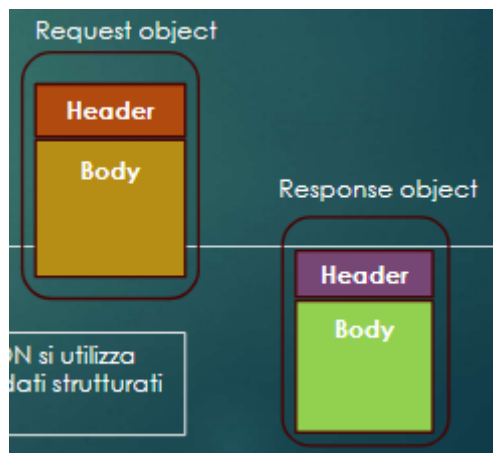
Trasporto dei Dati — Request e Response

Ogni comunicazione HTTP è composta da:

1. una **request** (**richiesta del client**)
2. una **response** (**risposta del server**).

Entrambe hanno la stessa struttura:

- **Header** — **contiene i metadati della comunicazione (tipo di contenuto, autenticazione, ecc.). È strutturato e segue regole precise.**
- **Body** — **contiene i dati veri e propri.**
 - Non segue regole di formato rigide: in passato si usava XML, oggi in ambito REST si usa **JSON**.



Come il Client Invia Dati al Server

Il client ha **tre modi** per inviare dati al server:

1. Query string — i dati vengono accodati direttamente all'URL dopo il carattere `?`, come coppie chiave-valore separate da `&`:



HTTP

```
1 http://ip:porta/path/risorsa?var1=aaa&var2=bbb
```

Tipicamente usata per parametri di filtro o ricerca, ad esempio `?autore=tolkien&prezzo=20`

2. Path variable — i dati vengono inseriti direttamente nel percorso dell'URL:



HTTP

```
1 http://ip:porta/path/risorsa/aaa/bbb
```

Tipicamente usata per identificare una risorsa specifica, ad esempio `/libri/42` per identificare il libro con `id = 42`.

3. Body della richiesta — i dati vengono inseriti nel body della request in formato JSON. È il metodo usato quando si devono inviare oggetti complessi con più campi, ad esempio per una `POST` di inserimento o una `PUT` di modifica.

Come il Server Invia Dati al Client

Il server ha **un solo modo** per inviare dati: inserendoli nel **body della response** in formato JSON.



JSON

```
1 {
2   "id": 1,
3   "titolo": "Informatica per tutti",
4   "autore": "Rob del",
5   "prezzo": 9.9
6 }
```

Nota - il collegamento con quanto già visto:

il `'LibroDTO'` che abbiamo costruito con JDBC **è esattamente la struttura che verrà serializzata in JSON e inserita nel body della response.**

Spring si occuperà di questa conversione automaticamente.

🛡 Terminologia: Body vs Payload

Payload e **body** vengono spesso usati come sinonimi nel contesto HTTP — entrambi indicano i dati trasportati nel corpo della richiesta o della risposta.

La distinzione, se vogliamo essere precisi, è sottile:

- **Body:**
 - è il termine tecnico HTTP
 - si riferisce alla parte della struttura del messaggio HTTP che contiene i dati
- **Payload:**
 - è un termine più generico che indica "il dato utile trasportato"
 - viene usato in molti contesti diversi (non solo HTTP) per indicare la parte significativa di un messaggio, escludendo i metadati

In pratica, quando un programmatore dice "metti i dati nel payload" o "metti i dati nel body" sta dicendo la stessa cosa.

Nel contesto di Spring e REST sentirai usare entrambi i termini in modo intercambiabile.

Status Code HTTP

Come abbiamo già visto nella lezione [Lezione 4 - Protocollo HTTP 2 parte al paragrafo "Status code"](#):

- il server è tenuto a restituire uno status code per ogni richiesta ricevuta, indicando l'esito dell'operazione.

I codici sono organizzati in famiglie da 100:

Famiglia	Significato	Esempi
1xx	Informazionale — la richiesta è in elaborazione	100 Continue
2xx	Successo — la richiesta è andata a buon fine	200 OK , 201 Created
3xx	Reindirizzamento — il client deve fare un'altra richiesta	301 Moved Permanently

Famiglia	Significato	Esempi
4xx	Errore lato client — la richiesta è malformata o non autorizzata	400 Bad Request , 401 Unauthorized , 404 Not Found
5xx	Errore lato server — il server ha fallito nell'elaborare una richiesta valida	500 Internal Server Error

⚠ **La distinzione tra 4xx e 5xx è fondamentale:**

- un errore 4xx significa che il problema è nella richiesta del client (dati mancanti, risorsa inesistente, mancanza di autenticazione) ,
- mentre un errore 5xx significa che la richiesta era corretta ma qualcosa è andato storto lato server — tipicamente un bug nel codice o un problema di infrastruttura .

HTTP Status Codes		
Level 200 (Success) 200 : OK 201 : Created 203 : Non-Authoritative Information 204 : No Content	Level 400 400 : Bad Request 401 : Unauthorized 403 : Forbidden 404 : Not Found 409 : Conflict	Level 500 500 : Internal Server Error 503 : Service Unavailable 501 : Not Implemented 504 : Gateway Timeout 599 : Network timeout 502 : Bad Gateway

Inoltre come ben sappiamo, questa regola che il server debba sempre restituire uno status code segue uno dei principi RESTful:

- ogni risposta HTTP deve essere autodescrittiva, cioè deve contenere tutte le informazioni necessarie per capire l'esito dell'operazione senza dover guardare altro.
- Lo status code è esattamente questo — un modo standardizzato e univoco per comunicare l'esito di ogni operazione.

In altri termini: un'API REST non dovrebbe mai restituire 200 OK con un messaggio di errore nel body — sarebbe contraddittorio e violerebbe questo principio. Se qualcosa è andato storto, lo status code deve rifletterlo.

Gli status code, insieme ai verbi HTTP, vengono introdotti formalmente al **livello 2** del Richardson Maturity Model:

- **il modello che misura il grado di maturità di una API REST.**
- **I verbi descrivono cosa vuoi fare sulla risorsa, gli status code descrivono com'è andata.**

Insieme formano il linguaggio standard di una API REST.

Raggiungere il livello 2 è considerato il minimo indispensabile per parlare di una vera API REST — il **livello 3 (HATEOAS)** rappresenta invece la cosiddetta **"Glory of REST"**

Il Formato JSON

JSON (JavaScript Object Notation) è:

- **il formato standard per lo scambio di dati nelle applicazioni REST.**
È simile all'XML ma molto più **leggero e leggibile**.

Un oggetto JSON è racchiuso tra `{ }` e le sue proprietà sono coppie **nome** : **valore** separate da virgole:

```
{ }  JSON

1  {
2      "id": 1,
3      "titolo": "Informatica per tutti",
4      "autore": "Rob del",
5      "prezzo": 9.9
6  }
```

Il valore di una proprietà può essere:

- **Numerico** — `"prezzo": 9.9`
- **Testo** — `"titolo": "Informatica per tutti"`
- **Booleano** — `"disponibile": true`
- **Array** — `"generi": ["informatica", "didattica"]`
- **Oggetto JSON annidato** — una chiave il cui valore è a sua volta un oggetto JSON:

```
{ }  JSON
```

```
1  {
2      "id": 1,
3      "titolo": "Informatica per tutti",
4      "autore": {
5          "nome": "Rob",
6          "cognome": "Del",
7          "nazionalità": "italiana"
8      }
9  }
```

Negoziazione del Formato

Sia client che server devono **dichiararsi reciprocamente** il formato dei dati che stanno usando o che si aspettano, tramite l'header della richiesta/risposta:

- Chi **invia** dati JSON deve specificare nell'header:

{ } JSON

```
1  content-type: application/json
```

- Chi **si aspetta** dati JSON deve specificare nell'header:

{ } JSON

```
1  accept: application/json
```

👁 Info

Se il client indica più formati accettabili e il server ne supporta almeno uno, il server sceglie il formato più appropriato tra quelli indicati.

Questo meccanismo si chiama content negotiation.